GenAi Analysis

Name of AI #1:
Gemini

Accessed:
On my computer by searching for the website.

Prompt:
Write a complete C++ program for EECS 348 Assignment 4 that solves five Sudoku puzzles using object-oriented programming and recursion. The program should use a class to represent and solve each Sudoku puzzle instead of using standalone functions. Each puzzle is a 9x9 grid where every row, column, and 3x3 block must contain the numbers 1 through 9 exactly once. The input files are named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt. Each file contains a partially completed puzzle, and underscores (_) represent empty cells. Use recursive backtracking to solve the puzzles. The program should find an empty cell, try each valid number from 1 to 9, and recursively keep solving the puzzle. If a choice does not lead to a solution, undo that choice and try another number. The program must find and print every valid solution, not just the first one. If a puzzle has no valid solution, print 'no solution found'. For each puzzle, print the filename, the original puzzle, and all of its solutions or the no-solution message. Make sure the program compiles and runs on the KU EECS Cycle Linux servers using the available C++ compiler.

Generated Code:
```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

// Class representing a Sudoku puzzle using Object-Oriented Programming principles
class Sudoku {
private:
    vector<vector<char>> grid; // 9x9 grid representing the Sudoku board
    int solutionCount;         // Tracks the number of valid solutions found

    // Helper method to check if placing a specific digit at (row, col) is valid
    bool isValid(int row, int col, char val) {
        // Check if the digit already exists in the current row or column
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == val) return false;
            if (grid[i][col] == val) return false;
        }
```

```
    // Check if the digit already exists in the corresponding 3x3 subgrid
    int startRow = row - (row % 3);
    int startCol = col - (col % 3);
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (grid[startRow + i][startCol + j] == val) return false;
        }
    }

    return val; // Valid placement
}

// Core recursive backtracking method to explore all possible configurations
void solveRecurse(int row, int col) {
    // Base Case: If row reaches 9, we have successfully filled all cells
    if (row == 9) {
        solutionCount++;
        cout << "Solution " << solutionCount << ":" << endl;
        printGrid();
        cout << endl;
        return; // Return to backtrack and search for alternative solutions
    }

    // Calculate coordinates for the next cell (move right, wrap to next row at the end)
    int nextRow = (col == 8) ? row + 1 : row;
    int nextCol = (col == 8) ? 0 : col + 1;

    // If the current cell is already filled with a given digit, skip to the next cell
    if (grid[row][col] != '_') {
        solveRecurse(nextRow, nextCol);
        return;
    }

    // Try digits '1' through '9' in the empty cell
    for (char val = '1'; val <= '9'; val++) {
        if (isValid(row, col, val)) {
            grid[row][col] = val;           // Make the choice
            solveRecurse(nextRow, nextCol); // Recursively explore further
            grid[row][col] = '_';           // Undo the choice (backtrack)
        }
    }
}
```

```
public:
    // Constructor initializes an empty 9x9 board
    Sudoku() {
        grid = vector<vector<char>>(9, vector<char>(9, '_'));
        solutionCount = 0;
    }

    // Loads a puzzle grid from a specified file
    bool loadPuzzle(const string& filename) {
        ifstream infile(filename);
        if (!infile.is_open()) {
            return false; // Return false if file cannot be opened
        }
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                infile >> grid[i][j];
            }
        }
        infile.close();
        solutionCount = 0;
        return true;
    }

    // Prints the 9x9 grid to the console formatted with spaces
    void printGrid() const {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                cout << grid[i][j] << (j == 8 ? "" : " ");
            }
            cout << endl;
        }
    }

    // Driver method to initiate solving and print results for a specific file
    void solve(const string& filename) {
        cout << "Filename: " << filename << endl;
        cout << "Original Puzzle:" << endl;
        printGrid();
        cout << endl;

        solutionCount = 0;
        solveRecurse(0, 0); // Start recursion from the top-left corner (0, 0)

        if (solutionCount == 0) {
```

```
            cout << "No solution found" << endl;
        } else {
            cout << "Solutions found: " << solutionCount << endl;
        }
        cout << "----------------------------------------" << endl;
    }
};

int main() {
    // Loop through puzzle1.txt to puzzle5.txt and solve each one sequentially
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";
        Sudoku puzzle;

        if (puzzle.loadPuzzle(filename)) {
            puzzle.solve(filename);
        } else {
            cout << "Filename: " << filename << endl;
            cout << "Error: Could not open file " << filename << endl;
            cout << "----------------------------------------" << endl;
        }
    }

    return 0;
}
```

Analysis:

The program works because it uses recursive backtracking to try different numbers and find every valid Sudoku solution. The isValid() method checks the row, column, and 3x3 block before trying a number to make sure it follows the Sudoku rules. The time complexity is O(9^N), where N is the number of empty cells. The program may need to try up to 9 numbers for each empty cell. The space complexity is “O(N)” because of the recursive calls, while the Sudoku grid itself takes a constant amount of space. The code is maintainable because it uses a Sudoku class to keep the puzzle and its methods organized, which makes it easier to understand than using only standalone functions. The program is organized and its comments are helpful and understandable, but the program could be improved by using return true; instead of return val; in the isValid() method and using constants instead of repeating the number 9 throughout the code.

Name of AI #2:

Copilot

Accessed:
On my computer by searching for the website.

Prompt:

Generated Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

/*
   Sudoku class

   This class stores a Sudoku puzzle and contains methods for:
   - Reading a puzzle from a file
   - Printing the puzzle
   - Checking whether a digit placement is valid
   - Solving the puzzle using recursive backtracking
*/
class Sudoku
{
private:
   int board[9][9];
   int solutionCount;

   /*
      Returns true if placing 'num' at board[row][col]
      does not violate Sudoku rules.
   */
   bool isValid(int row, int col, int num)
   {
      // Check row
      for (int c = 0; c < 9; c++)
      {
         if (board[row][c] == num)
         {
            return false;
         }
      }
```

```
    // Check column
    for (int r = 0; r < 9; r++)
    {
        if (board[r][col] == num)
        {
            return false;
        }
    }

    // Locate top-left corner of 3x3 block
    int startRow = row - row % 3;
    int startCol = col - col % 3;

    // Check 3x3 block
    for (int r = startRow; r < startRow + 3; r++)
    {
        for (int c = startCol; c < startCol + 3; c++)
        {
            if (board[r][c] == num)
            {
                return false;
            }
        }
    }

    return true;
}

/*
    Recursive depth-first search with backtracking.

    Algorithm:
    1. Find the next empty cell.
    2. Try digits 1 through 9.
    3. If valid, place the digit.
    4. Recursively solve remaining puzzle.
    5. Undo placement if it does not lead to a solution.
    6. Continue searching for all solutions.
*/
void solveRecursive()
{
    int row = -1;
    int col = -1;
    bool emptyFound = false;
```

```
        // Find first empty location
        for (int r = 0; r < 9 && !emptyFound; r++)
        {
            for (int c = 0; c < 9; c++)
            {
                if (board[r][c] == 0)
                {
                    row = r;
                    col = c;
                    emptyFound = true;
                    break;
                }
            }
        }

        // Base case:
        // no empty cells remain, therefore a solution is found
        if (!emptyFound)
        {
            solutionCount++;

            cout << "\nSolution " << solutionCount << ":\n";
            printBoard();

            return;
        }

        // Try digits 1-9
        for (int num = 1; num <= 9; num++)
        {
            if (isValid(row, col, num))
            {
                board[row][col] = num;

                solveRecursive();

                // Backtrack
                board[row][col] = 0;
            }
        }
    }

public:
```

```
/*
   Constructor initializes puzzle and solution count.
*/
Sudoku()
{
   solutionCount = 0;

   for (int r = 0; r < 9; r++)
   {
      for (int c = 0; c < 9; c++)
      {
         board[r][c] = 0;
      }
   }
}

/*
   Reads puzzle from input file.
   Returns true if successful.
*/
bool loadPuzzle(string filename)
{
   ifstream infile(filename);

   if (!infile)
   {
      return false;
   }

   string entry;

   for (int r = 0; r < 9; r++)
   {
      for (int c = 0; c < 9; c++)
      {
         infile >> entry;

         if (entry == "_")
         {
            board[r][c] = 0;
         }
         else
         {
            board[r][c] = stoi(entry);
```

```
            }
        }
    }

    infile.close();
    return true;
}

/*
    Prints puzzle in readable format.
*/
void printBoard() const
{
    for (int r = 0; r < 9; r++)
    {
        for (int c = 0; c < 9; c++)
        {
            cout << board[r][c] << " ";
        }
        cout << endl;
    }
}

/*
    Prints puzzle using underscores for empty spaces.
*/
void printOriginalPuzzle() const
{
    for (int r = 0; r < 9; r++)
    {
        for (int c = 0; c < 9; c++)
        {
            if (board[r][c] == 0)
            {
                cout << "_ ";
            }
            else
            {
                cout << board[r][c] << " ";
            }
        }
        cout << endl;
    }
}
```

```
    /*
       Starts solving process and reports results.
    */
    void solve()
    {
       solutionCount = 0;

       solveRecursive();

       if (solutionCount == 0)
       {
          cout << "\nNo solution found\n";
       }
       else
       {
          cout << "\nSolutions found: "
               << solutionCount << endl;
       }
    }
};

/*
   Main program

   Creates a Sudoku object for each puzzle file,
   loads the puzzle, displays it, and solves it.
*/
int main()
{
   const int NUM_PUZZLES = 5;

   for (int i = 1; i <= NUM_PUZZLES; i++)
   {
      string filename = "puzzle" + to_string(i) + ".txt";

      cout << "\n==================================";
      cout << "\nFile: " << filename;
      cout << "\n==================================\n";

      Sudoku puzzle;

      if (!puzzle.loadPuzzle(filename))
      {
```

```
            cout << "Error: Could not open "
                 << filename << endl;
            continue;
        }

        cout << "\nOriginal Puzzle:\n";
        puzzle.printOriginalPuzzle();

        cout << "\nSearching for solutions...\n";

        puzzle.solve();

        cout << endl;
    }

    return 0;
}
```

Analysis:

This program works well because it follows the Sudoku rules, finds all valid solutions, and handles puzzles that have no solution. The time complexity is O(9^N) in the worst case because it uses recursive backtracking to try different numbers, while the space complexity is O(N) because of the recursive calls. The code is maintainable because the Sudoku class keeps the puzzle and its methods organized. The variable names, separate methods, and comments also make the code easier to understand and fix if something goes wrong.

Improvements:

A few ways this program could be improved is by making the isValid() method use return true; instead of return val; to make the code clearer. The program also uses the number 9 in multiple places, so using a constant would make it easier to change later. Another improvement would be checking the input files before solving to make sure the puzzles are formatted correctly and follow the Sudoku rules. Finally, I would improve the way the board is printed so the user can properly see all 9 sections of the sudoku board.

Selection choice:

I'm choosing Copilot because it would be easier to make these improvements to the program. It could help update the code to make the board easier to read, use constants instead of repeating the number 9, and check the input files before solving the puzzles.